UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

FACULTY OF COMPUTING

SEMESTER 2/20252026

SECP3133 - HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 1: OPTIMIZING HIGH PERFORMANCE DATA PROCESSING FOR LARGE SCALE WEB CRAWLERS

GROUP *Reporter* TEAM MEMBERS:

NAME	MATRIC ID
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205

LECTURER:
Prof. Madya. Ts. Dr Mohd Shahizan bin Othman

SUBMITTED ON: 25 May 2026

Table of Contents

1.0 Introduction

1.1 Background

In the modern data-driven landscape, the ability to efficiently collect and process large volumes of structured information is a critical capability for any data engineering team. Web crawling and data pipeline optimization represent core skills in this domain, enabling organizations to extract practical information from publicly available sources at scale and produce valuable insight at the end of the process.

This project was undertaken as part of the High Performance Data Processing course, with the goal of designing, building, and optimizing a large-scale web crawler targeting a real Malaysian website. The project goes from basic data collection by applying high performance computing (HPC) techniques to improve pipeline efficiency, and evaluating the results using objective performance metrics.

1.2 Objectives

The primary objective of this project are:

1. To develop a functional web crawler using tools like beautifulsoup4, curl_cffi, beautifulsoup4, multiprocessing/multithreading and lxml that are capable of extracting more than 100,000 structured book records from MPH Online (mphonline.com), a leading Malaysian online bookstore.
2. To clean, transform and store the collected data in a structured and query ready format.
3. To implement at least two distinct HPC optimization techniques which are Pandas, Dask and PySpark to improve throughput and efficiency of the data pipeline.
4. To conduct a performance evaluation by comparing the baseline and optimized versions of the pipeline using metrics such as execution time, CPU utilization, memory consumption, and records processed per second.

1.3 Target Website

The selected data source for this web scraping project is MPH Online (https://mphonline.com/), one of Malaysia's most established book retailers with a strong presence. MPH Online hosts a comprehensive and well structured product catalogue covering over thousands of titles across diverse genres including fiction, non-fiction, academic texts, children's books and more.

The website was selected due to following reasons:

1. It contains a large amount and structured product catalogue to realistically meet the 100,000 or more record requirement across categories, subcategories, and individual product pages.
2. Product listings follow a consistent page structure, making them well-suited for systematic extraction.
3. The data available is publicly accessible and non-sensitive in nature, aligning with ethical data collection principles.

1.4 Data to be Extracted

Each record collected which is around 189,000 from MPH Online will capture the following structured fields:

Field	Description
Product id	A unique numerical identifier for the item
Title	The full title of the book.
Handle	A unique, URL-friendly string identifier (slug) representing the title, typically used in website URLs.
Vendor	The publisher or distributor responsible for the product
Author Name	The name of the author who wrote the book, formatted as Last Name, First Name.
Created Date	The timestamp indicates exactly when the product record was originally created.
Updated Date	The timestamp indicates when the product record was last modified or

	updated.
Published Date	The timestamp indicates when the product was officially published or made live.
Tags	Classification keywords or labels used to categorize the product
Book Description	A brief text summary, synopsis, or promotional blurb describing the book's content.
Source URL	The direct URL where the product is hosted.

2.0 System Design

2.1 Architecture Diagram

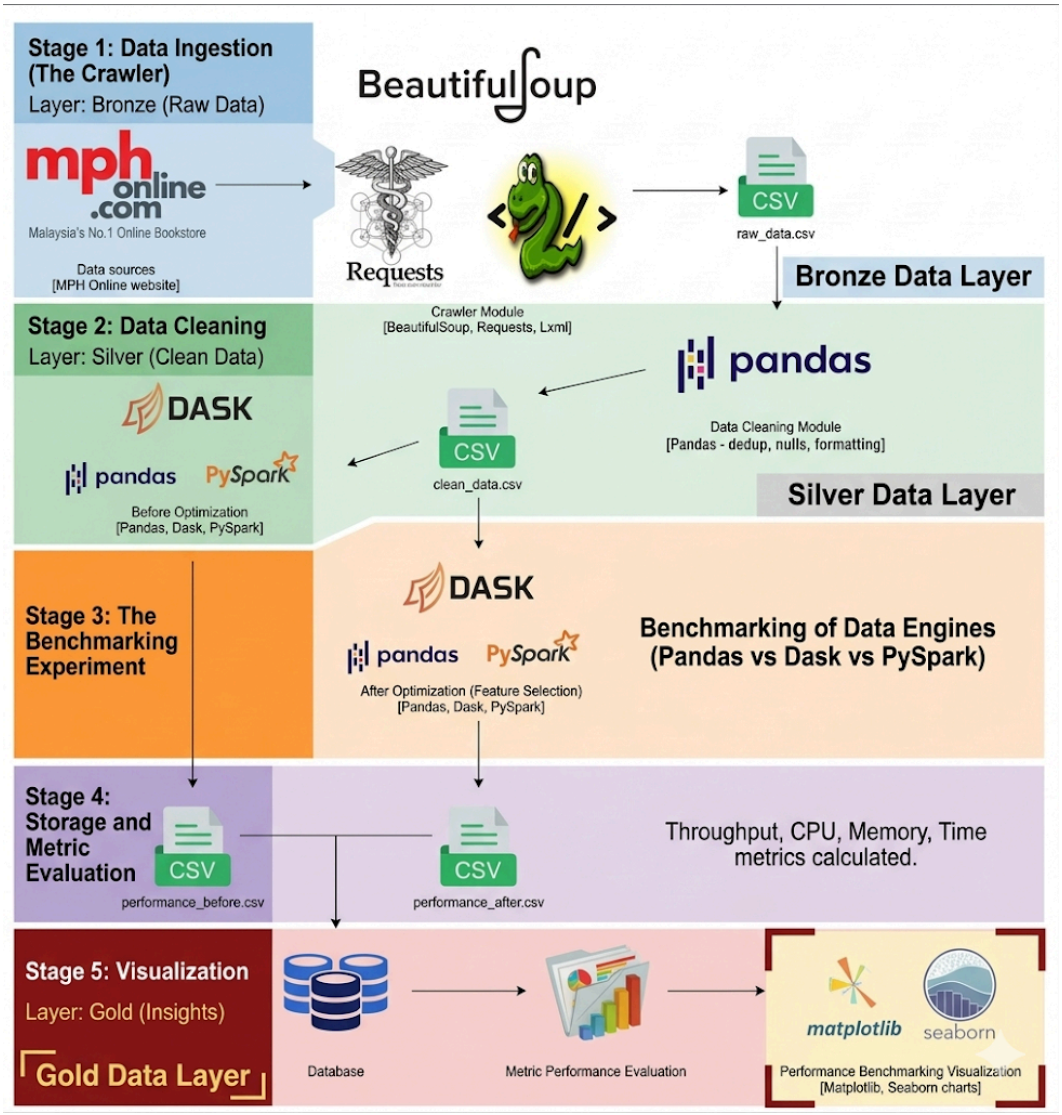

Figure 2.1 End-to-end Pipeline Architecture Design

Figure 2.1 is architecture design that has five crucial stages which are:

Stage 1: Data Ingestion (The Crawler)

The process kicks off with the Crawler Module, which targets the MPH Online bookstore as its primary data source. Using the Requests library, the system fetches the raw HTML from the website, which is then parsed by BeautifulSoup and lxml to extract specific details like book titles, prices, and categories. The final output of this stage is a file named raw_data.csv. In data

engineering terms, this is often referred to as your "Bronze" layer and it contains all the information needed, but it’s still dirty and unorganized.

Stage 2: Data Cleaning

Once the raw data is captured, it moves into the Data Cleaning Module. Here, Pandas take center stage as the primary engine for sanitization. The goal of this stage is to transform the messy raw file into a high-quality dataset by removing duplicate entries, handling missing (null) values, and ensuring that data types are correctly formatted. This results in the clean_data.csv, or the "Silver" layer, which serves as the reliable foundation for the benchmarking experiments that follow.

Stage 3: The Benchmarking Experiment

This is the heart of the architecture, where comparing how different frameworks handle data under different conditions. The process is split into two phases which are before and after optimization. In the first phase, the clean data is processed by three distinct engines including Pandas, Dask, and PySpark to establish a baseline performance. After this, a feature selection step is applied to optimize the dataset by keeping only the most relevant variables. The three engines then run again on this optimized data to see how the performance scales. Results from both phases are saved as performance_before.csv and performance_after.csv.

Stage 4: Storage and Metric Evaluation

After the engines have finished the work, the generated performance metrics are moved from temporary CSV files into a Database. Storing this data in a database rather than just flat files allows for better data integrity and easier querying of historical results. From here, the Metric Performance Evaluation takes place. This stage is where the "heavy lifting" of the analysis happens and calculating the total processing time, average CPU utilization, peak memory consumption and throughput that the feature selection optimization had on each of the three frameworks.

Stage 5: Visualization

The final stage of the pipeline transforms raw numbers into visual stories. Using Matplotlib and Seaborn, the system generates charts and graphs that represent the benchmarking results. These

visualizations make it easy to see at a glance which tool performed best and how much the optimization improved efficiency. This "Gold" layer of data provides the final insights needed to draw conclusions about whether a distributed system like PySpark or a parallel system like Dask is actually necessary for your specific dataset size.

2.2 Tools and Frameworks

Category	Tool / Library	Purpose
Web Crawling	Requests, BeautifulSoup, lxml	Fetching and parsing HTML pages
Data Storage	JSON, CSV	Storing raw and cleaned records
Data Cleaning	Pandas	Deduplication, null handling, formatting
Parallel Processing	Python multithreading / multiprocessing	Baseline optimization technique
Distributed Computing	PySpark, Dask	Other optimization technique
Benchmarking	psutil, time	Measuring CPU, memory, execution time, throughput
Visualization	Matplotlib, Seaborn	Performance comparison charts
Environment	Jupyter Notebook	Development and documentation
Version Control	GitHub	Code hosting and collaboration

This report outlines the technical and strategic justifications for selecting BeautifulSoup as the primary scraping engine for the MPH Online data pipeline. While there are several tools available in the Python ecosystem, BeautifulSoup was chosen for its specific balance of flexibility, ease of integration, and suitability for the target website's structure.

2.3 Team Roles and Responsibilities

Member	Role	Responsibilities
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	Group Leader & Data Engineer	Coordinates the team, oversees the crawler, manages data ingestion and storage, and ensures progress against the timeline.
AHMAD ADIB ZIKRI BIN A.MAZLAM	HPC Specialist & Documentation Lead	Designs and implements optimization techniques, runs benchmarks, and produces performance comparison charts.

Although roles are divided separately, all members highly contributed for the whole coding testing and review in this project.

3.0 Data Collection

3.1 Crawling Method

Step 1: Entry Point and Category Discovery

- The crawler starts at the MPH Online sitemap
- It collects all category and subcategory of URLs within sitemap.xml first before visiting each product pages
- Once collecting and confirming all the category or subcategory URLs, collect all product URLs especially on the <loc> location inside categories.

Step 2: Product Page Extraction

- For each product link found, the crawler visits the individual product page
- It then extracts the 11 target fields using BeautifulSoup by targeting specific HTML tags/classes.
- Each information product can be seen by using .json technique and it will display all the information in JSON form (see Figure A1 in Appendix A).

Step 3: Progressive Saving

- Records are saved in batches (e.g. every 500 or 1000 records) to a JSON or CSV file.
- This prevents data loss if the crawler crashes mid-run.
- All the progress data will save into a file named raw_data.csv.

3.2 Baseline Crawler Design

- Built using requests, BeautifulSoup and lxml.
- Processes one URL at a time sequentially.
- Includes basic error handling (try/except for timeouts, HTTP errors, missing fields).
- Crawl delay of 1.5 seconds minimum and 3.5 seconds maximum between requests.

3.3 Number of Records Collected

Number of rows (raw_data.csv)	~189,456 rows
Number of rows (cleaned_data.csv)	~189,398 rows
Number of columns	11 columns

List of columns after cleaning: id, title, handle, vendor, author, created_at, updated_at, published_at, tags, description, and source_url.

3.4 Ethical Considerations

robots.txt Compliance

Prior to beginning any data collection, the group reviewed the official robots.txt file located at https://mphonline.com/robots.txt. The file follows Shopify's standard crawler policy. Key findings relevant to this project:

- Allow: / — All public pages are crawlable by default
- The crawler specifically targeted /collections/ and /products/ paths, both of which are explicitly permitted
- The following paths were strictly avoided as they are disallowed:
    - /admin, /cart/, /checkout, /checkouts/, /orders, /account
    - /services, /sf_*, /cdn/wpm/*.js
    - Any URL containing sort_by, filter combinations (filter*&*filter), or preview parameters (preview_theme_id, preview_script_id) — these are disallowed as crawl traps
    - AJAX endpoints such as /cart.js and /recommendations/products

The crawler was designed to only access product and collection pages, fully respecting these boundaries.

Crawl Delay

No explicit Crawl-delay directive was specified in the robots.txt file. However, the group voluntarily applied a delay of 1.5 to 3.5 seconds between requests to avoid placing unnecessary load on the server and to mimic reasonable human browsing behavior.

No Login or Bypass

All data collected was from publicly accessible product and collection pages only. No authentication was attempted, no account-restricted pages were accessed, and no checkout or transactional flows were triggered. The account/login path was avoided entirely despite being technically allowed, as it was irrelevant to the project scope.

Shopify Platform Awareness

MPH Online operates on the Shopify platform. The robots.txt file explicitly notes that agents and automated tools should use the UCP/MCP endpoints for cart and checkout interactions. Since this project involves read-only data collection from public product pages only with no cart, checkout, or payment interactions and this restriction does not apply to our crawler.

Terms of Service

The group acknowledged Shopify's Terms of Service referenced in the robots.txt (https://www.shopify.com/legal/terms). All collected data is used strictly for academic and analytical purposes within the scope of this course assignment, and will not be redistributed commercially or published publicly.

Data Privacy

The data extracted consists entirely of publicly listed product information for example book titles, authors, descriptions and tags. No personal user data, transaction records, or private information was collected at any point during the crawling process.

4.0 Data Processing

4.1 Data Structure

The raw dataset was stored in CSV format (raw_data.csv) and loaded into a Pandas DataFrame for processing. Upon loading, the dataset contained the following key columns relevant to this project:

Raw Column Name	Issue Identified	Action Taken
body_html	Contains raw HTML markup	Stripped to plain text, renamed to description
tags	Mixed irrelevant and relevant tags	Filtered to retain only structured prefixes
product_type	Misleading column name	Renamed to author
id	Stored as numeric type	Converted to string type
title	Some rows contain numbers or dates instead of book titles	Invalid rows removed

After cleaning, the finalized columns of interest are id, title, author, description, tags, and any additional fields retained from the crawl such as vendor, published_at, source_url and others.

4.2 Cleaning Methods and Transformation Steps

The cleaning pipeline was executed sequentially across six steps as follows:

Step 1: Load the Dataset

The raw CSV file was loaded into a Pandas DataFrame using pd.read_csv(). Upon loading, the shape of the dataset (total rows and columns) was printed as a confirmation, and the three primary columns to be cleaned (body_html, tags, product_type) were previewed to verify they were present and correctly parsed. Any columns found missing at this stage were flagged with a warning before proceeding (see Figure A2 in Appendix A).

Step 2: Strip HTML Tags from body_html and Rename to description

The body_html column contained raw HTML markup sourced directly from MPH Online's product pages, including tags such as <p>, , , <h5>, and
 along with inline styles and attributes. This rendered the field unreadable as plain text and unsuitable for any downstream text analysis.

A custom function strip_html() was implemented using Python's re module to remove all HTML tags via regex substitution. Residual whitespace and newline characters were subsequently collapsed into single spaces and stripped from the edges of each value. Non-string values such as NaN were handled gracefully by returning an empty string. The column was then renamed from body_html to description to better reflect its content (see Figure A3 in Appendix A).

Step 3: Filter the tags Column

Each product's tags field contained a comma-separated string of multiple tag values, many of which were irrelevant to the project scope like colour tags, brand marketing tags, or internal system tags. Retaining all tags would introduce noise into the dataset and complicate any categorical analysis.

A custom function filter_tags() was implemented to parse each comma-separated tag string, evaluate each individual tag against a set of allowed prefixes, and retain only those that matched. The three accepted prefixes were:

Class Code_ : representing classification codes
Class_ : representing general class categories
Department_ : representing the department or genre grouping

Filtered tags were rejoined into a comma-separated string and written back to the tags column, overwriting the original values. Empty or non-string values were handled by returning an empty string (see Figure B1 in Appendix B).

Step 4: Rename product_type to author

The Shopify platform uses the generic column name product_type to store a classification field. In the context of MPH Online's book catalogue, this field holds the author's name. To improve

readability and semantic clarity, the column was renamed from product_type to author. No transformations were applied to the cell values themselves, only the column header was changed.

Step 5: Convert id Column from Numeric to String

By default, Pandas infers numeric-looking columns such as id as integer or float types. Storing an identifier field as a numeric type introduces several risks: leading zeros may be silently dropped, the value may be treated as a number subject to arithmetic operations, and downstream systems expecting a string identifier may receive incorrect input.

The id column was converted to string type using .astype(str). Any NaN values were first filled with empty strings prior to conversion to prevent them from being rendered as the literal string "nan", which was subsequently replaced with a true empty string (see Figure B2 in Appendix B).

Step 6: Remove Invalid Title Rows

A data quality issue was discovered in the title column where certain rows contained purely numeric values like 7.7157E+12 or date strings like 1/7/1914 0:00 or 11-Sep-01 instead of actual book titles. These rows were identified as corrupt or misaligned entries from the crawling stage and were removed from the dataset.

A custom function is_number_or_date() was implemented to detect such values. It first attempts to parse the value as a float to catch integers, decimals, and scientific notation. If that fails, it attempts to parse the value as a date using dateutil.parser, with an additional guard requiring the presence of a digit to prevent plain words from being incorrectly matched. Rows where the title matched either condition were dropped and the index was reset (see Figure B3 in Appendix B).

Step 7: Validation and Save

Before saving, a final sanity check was performed to confirm that all expected new column names (description, tags, author, id) were present with the correct data types, and that all old column names (body_html, product_type) had been successfully removed. The cleaned DataFrame was then exported to cleaned_data.csv without the default Pandas index column. The final row count and column count were printed as confirmation.

5.0 Optimization Technique

5.1 Description of the Method

For optimization of this project, the group used the technique Feature Selection. In the baseline approach which is not optimized, the data pipeline loads the entire dataset where it applies transformations to all columns such as converting numeric data to string before dropping the unwanted features. So, this baseline technique is inefficient as system resources are wasted while processing data that is most likely discarded.

To solve this problem, as mentioned earlier, the group used Feature Selection. The data pipeline will choose only the necessary target columns at the beginning. This will reduce the data volume stored in memory before performing any transformations. To know the difference of both method, the group applied both method across three frameworks of data processing which are Pandas, Dask, and PySpark.

5.2 Supporting Code Excerpts

5.2.1 Pandas

Figure 5..1 show baseline where it load all data, transform it, and subset at the end.

```
def pandas_baseline():
    # Load all data explicitly as strings
    df = pd.read_csv(DATA_FILE, dtype=str)

    # Baseline: Apply string uppercase transformation to ALL columns first
    for col_name in df.columns:
        df[col_name] = df[col_name].str.upper()

    # Late feature selection (dropping columns at the very end)
    df = df[features_to_keep]
    return len(df)
```

Figure 5.1 Baseline Method for Pandas

Figure 5.2 show optimized where it use *usecols* to load only the necessary features into memory.

```
def pandas_optimized():
    # Early feature selection: only load the columns we actually want to keep
    df = pd.read_csv(DATA_FILE, usecols=features_to_keep, dtype=str)

    # Optimized: Apply string transformation ONLY to the selected columns
    for col_name in df.columns:
        df[col_name] = df[col_name].str.upper()

    return len(df)
```

Figure 5.2 Optimized Method for Pandas

5.2.2 Dask

Figure 5.3 show baseline where it load all data, transform it, and subset at the end.

```
def dask_baseline():
    # Load all columns as string data
    ddf = dd.read_csv(DATA_FILE, dtype=str)

    # Baseline: Uppercase conversion across ALL string columns
    for col_name in ddf.columns:
        ddf[col_name] = ddf[col_name].str.upper()

    ddf = ddf[features_to_keep]
    return len(ddf.compute())
```

Figure 5.3 Baseline Method for Dask

Figure 5.4 show optimized where it use *usecols* to load only the necessary features into memory.

```
def dask_optimized():
    # Early feature selection during initial CSV load
    ddf = dd.read_csv(DATA_FILE, usecols=features_to_keep, dtype=str)

    # Optimized: Uppercase conversion ONLY on selected features
    for col_name in ddf.columns:
        ddf[col_name] = ddf[col_name].str.upper()

    return len(ddf.compute())
```

Figure 5.4 Optimized Method for Dask

5.2.3 PySpark

Figure 5.5 show baseline where it load all data, transform it, and subset at the end.

```
def pyspark_baseline():
    # Load data without inferring schemas (defaults to string type)
    df = spark.read.csv(DATA_FILE, header=True, inferSchema=False)

    # Baseline: Perform heavy uppercase loop across every column in the dataset
    for col_name in df.columns:
        df = df.withColumn(col_name, upper(df[col_name]))

    # Late feature selection
    df = df.select(*features_to_keep)
    return df.count()
```

Figure 5.5 Baseline Method for PySpark

Figure 5.6 show optimized where it load only the necessary features into memory.

```
def pyspark_optimized():
    df = spark.read.csv(DATA_FILE, header=True, inferSchema=False)

    # Early feature selection: narrow down column volume immediately
    df = df.select(*features_to_keep)

    # Optimized: Only modify the columns we care about
    for col_name in df.columns:
        df = df.withColumn(col_name, upper(df[col_name]))

    return df.count()
```

Figure 5.6 Optimized Method for PySpark

6.0 Performance Evaluation

To evaluate the effectiveness of Feature Selection, the pipeline was run three times for each frameworks with average performance records. The performance benchmarks are Total Execution Time (seconds), Average CPU Utilization (%), Peak Memory Usage (MB), and Throughput (Records processed per second).

6.1 Before Optimization

Figure 6.1 show the average performance of the pipeline using baseline method.

```
=== AVERAGE PERFORMANCE (BEFORE OPTIMIZATION) ===
```

	Framework	Total_Time_sec	Avg_CPU_percent	Peak_Memory_MB	Throughput_rec_per_sec
3	Pandas	2.2281	93.91	1379.19	85015.13
7	Dask	2.2576	112.87	1137.43	85740.10
11	PySpark	1.6110	9.76	1076.33	249098.09

Figure 6.1 Average Performance of Baseline Method

6.2 After Optimization

Figure 6.2 show the average performance of the pipeline using Feature Selection method.

```
=== AVERAGE PERFORMANCE (AFTER OPTIMIZATION) ===
```

	Framework	Total_Time_sec	Avg_CPU_percent	Peak_Memory_MB	Throughput_rec_per_sec
3	Pandas	1.5428	92.06	1332.43	122796.16
7	Dask	1.4505	116.79	1415.54	130787.18
11	PySpark	0.3497	7.45	1221.14	541946.42

Figure 6.2 Average Performance of Feature Selection Method

As can be seen, after the optimization executes faster than baseline. But, that does not mean it is good for all hardware. For average CPU and peak memory usage barely show differences. So, to see more, the are graph for this comparison

6.3 Framework Performance Comparison

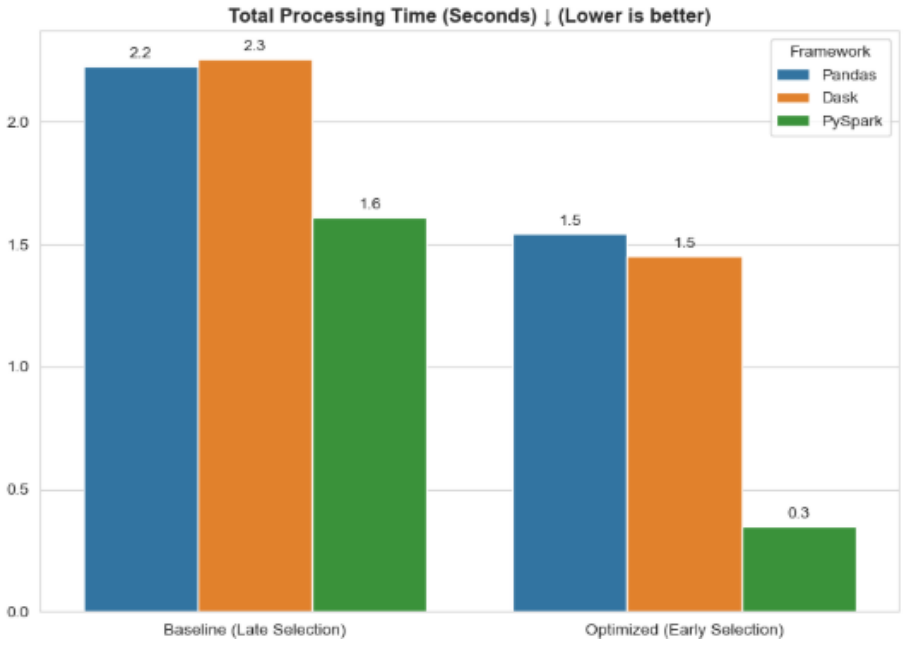

Figure 6.3 Total Processing Time (Seconds)

Figure 6.3 show the total processing time for each framework with different methods. As expected, Optimized execute faster but PySpark show that it is the fastest framework.

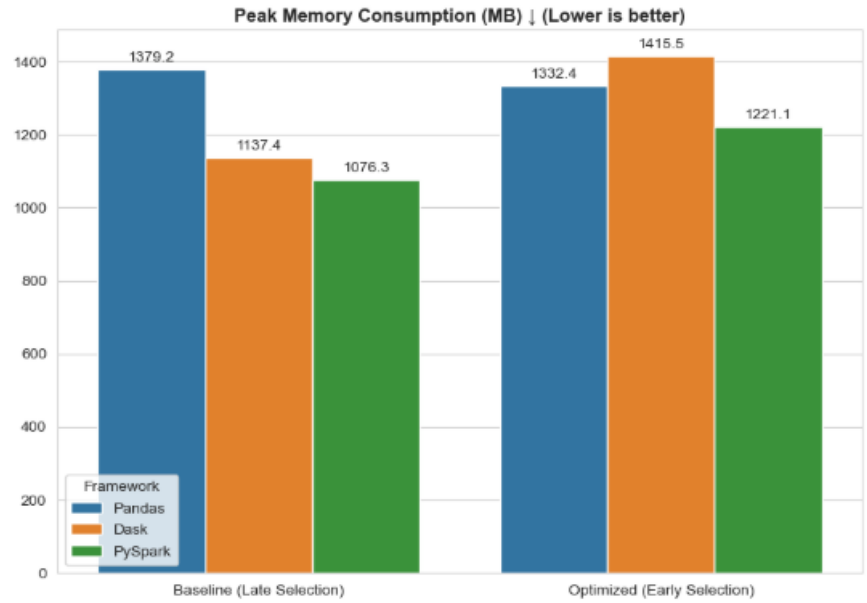

Figure 6.4 Peak Memory Consumption (MB)

Figure 6.4 show peak memory consumption in MB for each frame work with different method. The Baseline less use of memory because no selection for columns with PySpark at it best.

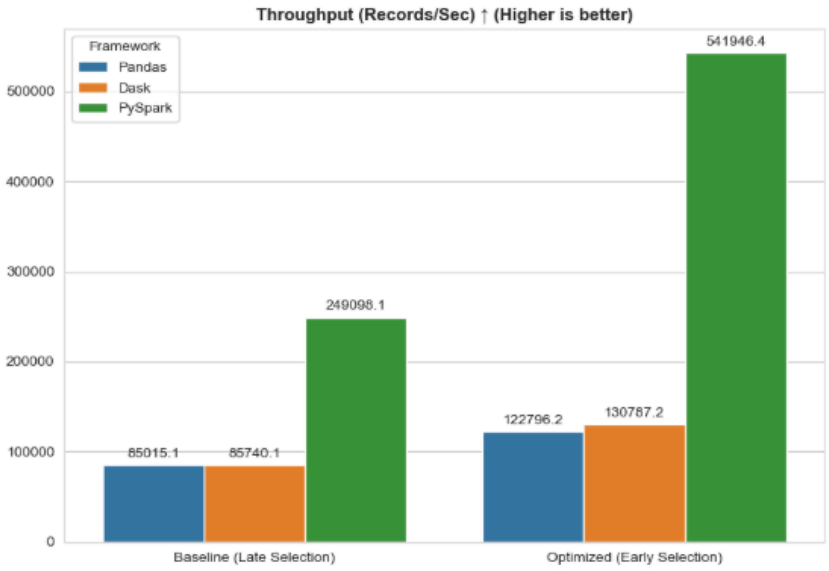

Figure 6.5 Throughput(Records/Sec)

Figure 6.5 show throughput that records per seconds for each frame work with different method. After optimization, it show better records with PySpark vastly outperforms the other two frameworks.

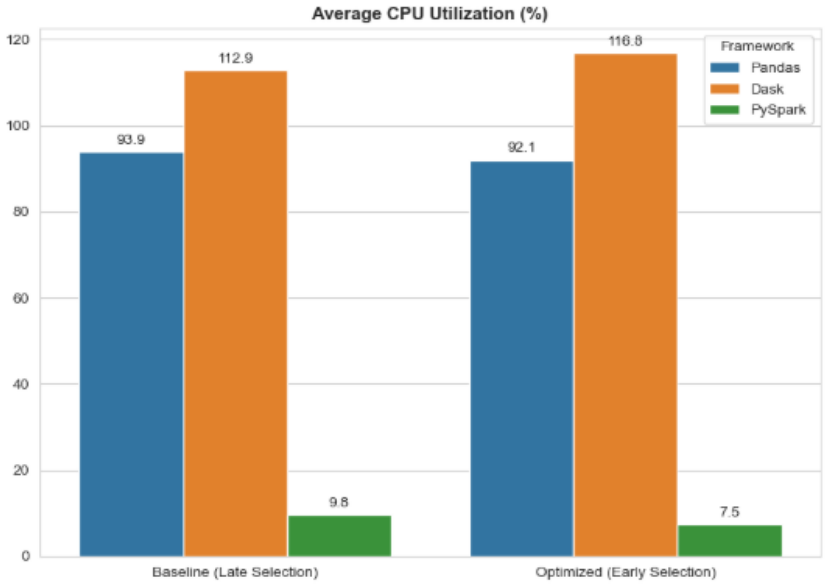

Figure 6.6 Average CPU Utilization (%)

Figure 6.6 show average CPU usage for each frame work with different methods. Pyspark frameworks show big difference from other framework but Optimized use less CPU than baseline

7.0 Challenges and Limitations

7.1 Challenges

Data Scraping Unsuccessful

While finding website to be scrap, not all website allow scraping without getting banned. For this group project, the scrap failed even not reaching half. The website block the IP address that the group used. So, it is hard to complete the scraping.

Data Quality Inconsistencies

The raw data extracted from the source website contained numerous inconsistencies. For example, the *body_html* column was heavily cluttered with complex HTML tags <*p*>, <*div*>, <*strong*> making it unreadable without custom regex filtering.

Data Fields Misleading

Data fields inherited from the Shopify platform structure required extensive evaluation and remapping; for example, the *product_type* column actually contained the author's name, which could have skewed downstream analytics if not manually identified and renamed.

Data Noises

For *tags* column, large amounts of pointless marketing and color tags were combined. To avoid noise in the final dataset, a special string-matching tool was developed to isolate only relevant classification and department prefixes

7.2 Limitations

Crawling Execution Time

In order to follow ethical scraping guidelines, a voluntary crawl delay of 1.5 to 3.5 seconds was enforced between requests.Given the massive scale of the scrape within 189,456 raw records, this delay significantly extended the total duration of the data ingestion phase.

Hardware Bottlenecks

While the Feature Selection optimization technique successfully reduced execution time, the benchmarking revealed that average CPU and peak memory usage barely showed differences on the hardware used. In fact, the baseline method sometimes exhibited less memory usage because it lacked the column selection overhead. So, it indicates that local hardware limits can sometimes restrict the full potential of large-scale data engines.

8.0 Conclusion and Future Work

In the end, the project was successful and managed to be finished. Including designing, building, and optimizing, the group manages to perform a large scale web scraping from MPH Online. The pipeline efficiently ingested, cleaned, and transformed raw HTML data into a highly structured, query-ready dataset. By conducting a benchmarking experiment across Pandas, Dask, and PySpark, the project demonstrated the value of High-Performance Computing (HPC) frameworks. The implementation of the Feature Selection optimization technique resulted in faster execution times and reduced CPU utilization compared to the baseline. Among the evaluated engines, PySpark emerged as the superior framework for this dataset size, vastly outperforming both Pandas and Dask in total processing time and throughput.

For future work, some enhancements can be considered. While PySpark showcased the highest throughput locally, migrating the pipeline to a distributed cloud environment can produce even better performance metrics for larger datasets. On the other hand, The current crawler could be expanded to target other major online bookstores. By capturing pricing and availability data across multiple platforms, the pipeline could support comparative analytics and market price tracking. Lastly, integrating an orchestration tool, such as Apache Airflow would enable scheduled and recurring crawls to keep the dataset updated with new book releases automatically without needing manual intervention.

9.0 References

1. GeeksforGeeks. (2026, January 13). *Implementing web scraping in Python with BeautifulSoup*. https://www.geeksforgeeks.org/python/implementing-web-scraping-python-beautiful-soup/

2. GeeksforGeeks. (2025, July 15). *Dask in Python*. https://www.geeksforgeeks.org/python/introduction-to-dask-in-python/

3. MPH Online. (n.d.). *Home*. https://mphonline.com

4. lxml. (2026, May 18). *lxml – XML and HTML with Python*. https://lxml.de/

5. W3Schools. (n.d.). *Pandas tutorial*. https://www.w3schools.com/python/pandas/default.asp

6. GeeksforGeeks. (2025, July 18). *PySpark tutorial*. https://www.geeksforgeeks.org/python/pyspark-tutorial/

7. W3Schools. (n.d.). *Python requests module*. https://www.w3schools.com/python/module_requests.asp

APPENDIX A

Link to the repository:

https://github.com/drshahizan/HPDP/tree/main/2526/project/p1/Reporter

Link dataset: data

Figure A1 shows the example of product information using .json technique

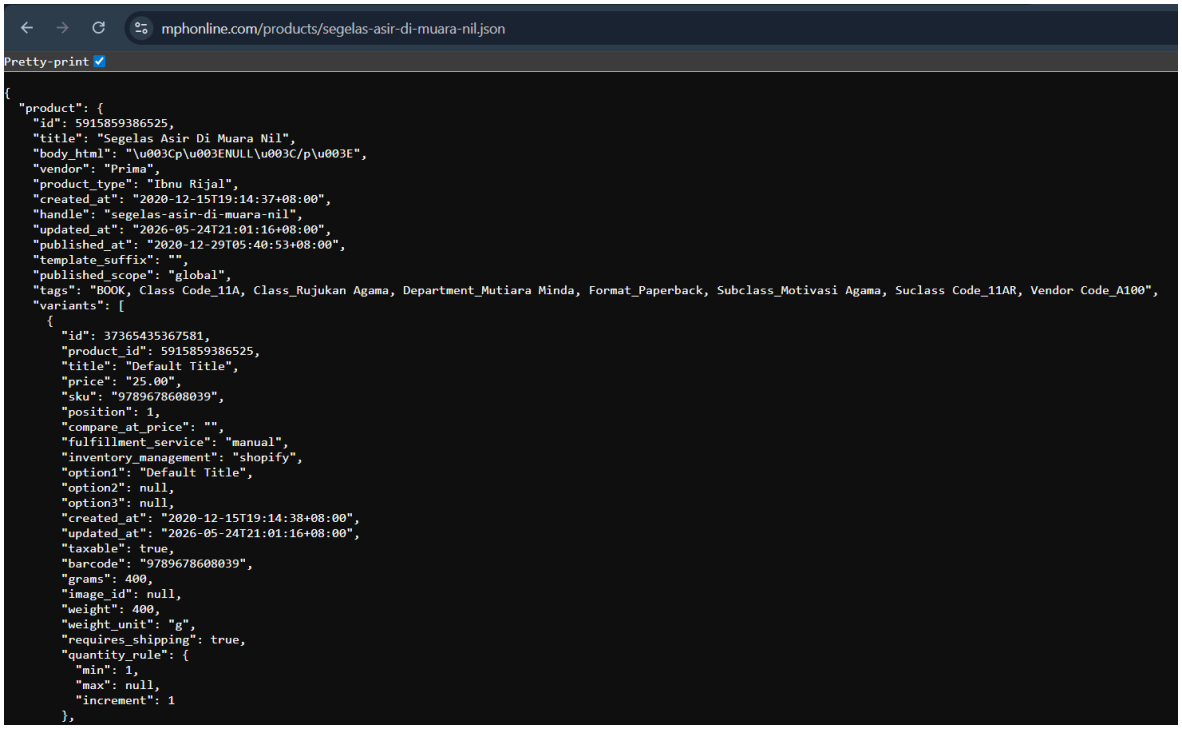

Figure A2 shows the code to load the dataset (raw_data.csv)

```
# ── Configuration ──────────────────────────────────────────────
FILE_PATH   = "../data/raw_data.csv"
OUTPUT_PATH = "cleaned_data.csv"

# ── Load ───────────────────────────────────────────────────────
df = pd.read_csv(FILE_PATH)

print(f"Dataset loaded — {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"\nColumns present: {df.columns.tolist()}")
```

FIgure A3 shows how to strip HTML Tags from body_html and rename it

```
def strip_html(raw: str) -> str:
    """Remove all HTML tags and normalise whitespace."""
    if not isinstance(raw, str):
        return ""                          # handle NaN / non-string values
    text = re.sub(r"<[^>]+>", " ", raw)    # replace every <tag …> with a space
    text = re.sub(r"\s+", " ", text).strip()  # collapse multiple spaces / newlines
    return text
```

APPENDIX B

Figure B1 shows how to filter column tags

```python
def filter_tags(tag_string: str) -> str:
    """Keep only tags matching the allowed prefixes."""
    if not isinstance(tag_string, str) or tag_string.strip() == "":
        return ""                                   # handle NaN / empty cells
    tags = [t.strip() for t in tag_string.split(",")]
    kept = [t for t in tags if t.startswith(KEEP_PREFIXES)]
    return ", ".join(kept)
```

Figure B2 shows conversion column from numeric to String

```python
# Convert id to string, filling any NaN with an empty string first
df["id"] = df["id"].fillna("").astype(str)

# If the column was numeric, astype(str) may produce '0' for NaN-filled rows;
# replace those placeholders back with an empty string.
df["id"] = df["id"].replace("nan", "")
```

Figure B3 shows the removal invalid title name

```python
def is_number_or_date(value: str) -> bool:
    """Return True if the value is purely a number (int/float) or a date string."""
    val = str(value).strip()

    if val == "" or val.lower() == "nan":
        return False

    # Check if it's a pure number (int, float, scientific notation)
    try:
        float(val)
        return True
    except ValueError:
        pass

    # Check if it's a pure date string (e.g. "1/7/1914 0:00", "11-Sep-01")
    try:
        date_parse(val, fuzzy=False)
        # Extra guard: must contain a digit and NOT be a plain word
        if re.search(r"\d", val):
            return True
    except Exception:
        pass

    return False
```